

```

import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# 1. Load the 'california_housing_train.csv' dataset
df = pd.read_csv('/content/sample_data/california_housing_train.csv')

print("DataFrame head:")
print(df.head())
print("\nDataFrame info:")
df.info()

# 2. Separate the target variable ('median_house_value') from the features
X = df.drop('median_house_value', axis=1)
y = df['median_house_value']

print(f"\nFeatures shape: {X.shape}")
print(f"Target shape: {y.shape}")

# 3. Select only the numeric features
numeric_features = X.select_dtypes(include=np.number)
print(f"\nNumeric features selected: {list(numeric_features.columns)}")

# 4. Handle missing values in the numeric features using SimpleImputer (strategy='mean')
imputer = SimpleImputer(strategy='mean')
X_imputed = pd.DataFrame(imputer.fit_transform(numeric_features), columns=numeric_features.columns)

print("\nMissing values after imputation:")
print(X_imputed.isnull().sum().sum())

# 5. Apply feature scaling to the imputed numeric features using StandardScaler
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X_imputed), columns=X_imputed.columns)

print("\nScaled features head:")
print(X_scaled.head())

```

```

DataFrame head:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -114.31    34.19             15.0         5612.0         1283.0
1    -114.47    34.40             19.0         7650.0         1901.0
2    -114.56    33.69             17.0          720.0          174.0
3    -114.57    33.64             14.0        1501.0          337.0
4    -114.57    33.57             20.0        1454.0          326.0

```

```

   population  households  median_income  median_house_value
0      1015.0       472.0         1.4936         66900.0
1      1129.0       463.0         1.8200         80100.0
2       333.0       117.0         1.6509         85700.0
3       515.0       226.0         3.1917         73400.0
4       624.0       262.0         1.9250         65500.0

```

```

DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999

```

Data columns (total 9 columns):

#	Column	Non-Null	Count	Dtype
0	longitude	17000	non-null	float64
1	latitude	17000	non-null	float64
2	housing_median_age	17000	non-null	float64
3	total_rooms	17000	non-null	float64
4	total_bedrooms	17000	non-null	float64
5	population	17000	non-null	float64
6	households	17000	non-null	float64
7	median_income	17000	non-null	float64
8	median_house_value	17000	non-null	float64

dtypes: float64(9)

memory usage: 1.2 MB

Features shape: (17000, 8)

Target shape: (17000,)

Numeric features selected: ['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income', 'median_house_value']

Missing values after imputation:

0

Scaled features head:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	2.619365	-0.671520	-1.079671	1.361695	1.764204	
1	2.539569	-0.573264	-0.761872	2.296608	3.230441	
2	2.494683	-0.905463	-0.920772	-0.882462	-0.866956	
3	2.489696	-0.928857	-1.159121	-0.524186	-0.480230	
4	2.489696	-0.961609	-0.682422	-0.545747	-0.506328	

	population	households	median_income
0	-0.361184	-0.075998	-1.252543
1	-0.261865	-0.099404	-1.081483
2	-0.955354	-0.999252	-1.170105
3	-0.796793	-0.715774	-0.362600
4	-0.701830	-0.622148	-1.026454

```
from sklearn.model_selection import train_test_split
```

```
# Split the preprocessed data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
```

```
print(f"X_train shape: {X_train.shape}")
```

```
print(f"X_test shape: {X_test.shape}")
```

```
print(f"y_train shape: {y_train.shape}")
```

```
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (13600, 8)
```

```
X_test shape: (3400, 8)
```

```
y_train shape: (13600,)
```

```
y_test shape: (3400,)
```

```
from sklearn.linear_model import LinearRegression
```

```
# Instantiate a Linear Regression model object
```

```
linear_model = LinearRegression()
```

```
# Fit the model to the training data
linear_model.fit(X_train, y_train)

print("Linear Regression model trained successfully.")
```

Linear Regression model trained successfully.

```
from sklearn.linear_model import Lasso

# Instantiate a Lasso Regression model object with a default alpha value
lasso_model = Lasso(alpha=1.0, random_state=42)

# Fit the model to the training data
lasso_model.fit(X_train, y_train)

print("Lasso Regression model trained successfully with alpha=1.0.")
```

Lasso Regression model trained successfully with alpha=1.0.

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Evaluate Linear Regression model
print("\n--- Linear Regression Model Evaluation ---")
# Make predictions on the test set
y_pred_linear = linear_model.predict(X_test)

# Calculate metrics
mse_linear = mean_squared_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)
r2_linear = r2_score(y_test, y_pred_linear)

# Print metrics
print(f"Mean Squared Error (MSE): {mse_linear:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_linear:.2f}")
print(f"R-squared (R2) Score: {r2_linear:.2f}")
```

```
--- Linear Regression Model Evaluation ---
Mean Squared Error (MSE): 4634658406.22
Root Mean Squared Error (RMSE): 68078.33
R-squared (R2) Score: 0.66
```

```
print("\n--- Lasso Regression Model Evaluation ---")
# Make predictions on the test set
y_pred_lasso = lasso_model.predict(X_test)

# Calculate metrics
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mse_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

# Print metrics
print(f"Mean Squared Error (MSE): {mse_lasso:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_lasso:.2f}")
print(f"R-squared (R2) Score: {r2_lasso:.2f}")
```

```
--- Lasso Regression Model Evaluation ---
Mean Squared Error (MSE): 4634691896.95
Root Mean Squared Error (RMSE): 68078.57
R-squared (R2) Score: 0.66
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Extract the coefficients
linear_coef = linear_model.coef_
lasso_coef = lasso_model.coef_

# 2. Create a DataFrame to store the feature names and coefficients
coefficients_df = pd.DataFrame({
    'Feature': X_scaled.columns,
    'Linear_Regression': linear_coef,
    'Lasso_Regression': lasso_coef
})

# Reshape the DataFrame for plotting (melt it)
coefficients_melted = coefficients_df.melt(id_vars='Feature', var_name='Model', \

# 3. Create a bar plot to visualize the coefficients
plt.figure(figsize=(12, 7))
sns.barplot(x='Feature', y='Coefficient', hue='Model', data=coefficients_melted)
plt.xlabel('Feature Name')
plt.ylabel('Coefficient Value')
plt.title('Comparison of Linear Regression and Lasso Regression Coefficients')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Regression Model')
plt.tight_layout()
plt.show()
```



